

CSE6001

BIG DATA FRAMEWORKS

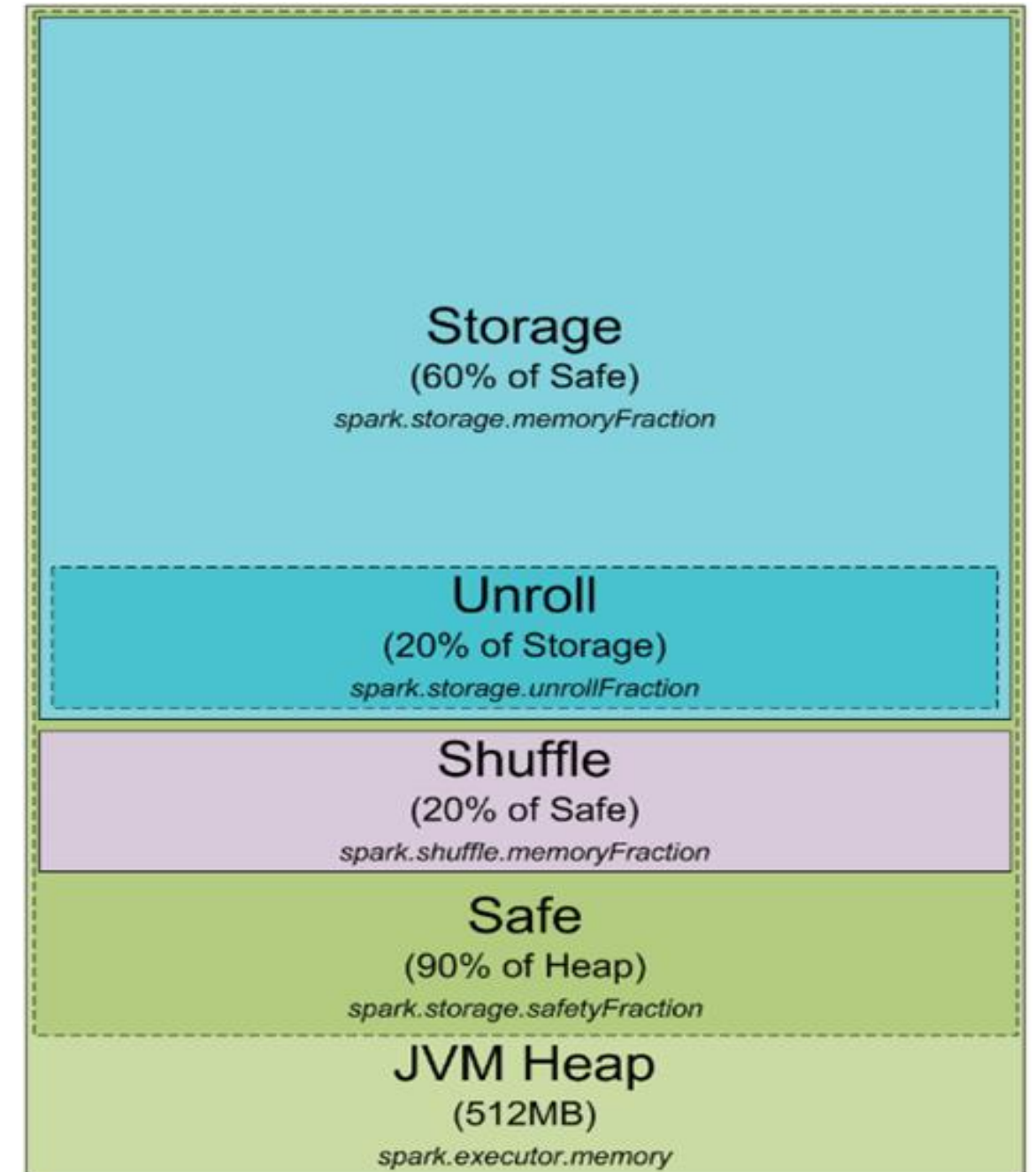
Module - 5

RDD

Prof. Lokeshkumar R

Spark Memory Management

- Memory utilization is essential in Spark (Caching)
- Spark process is a JVM process



Spark Programming Model

- High-Level coding to build a workflow (Scala)
- Code compiles to distributed parallel operations
- Two Abstraction Units
 - **RDDs: Resilient Distributed Datasets**
 - **Parallel Operations**

RDD: Concept

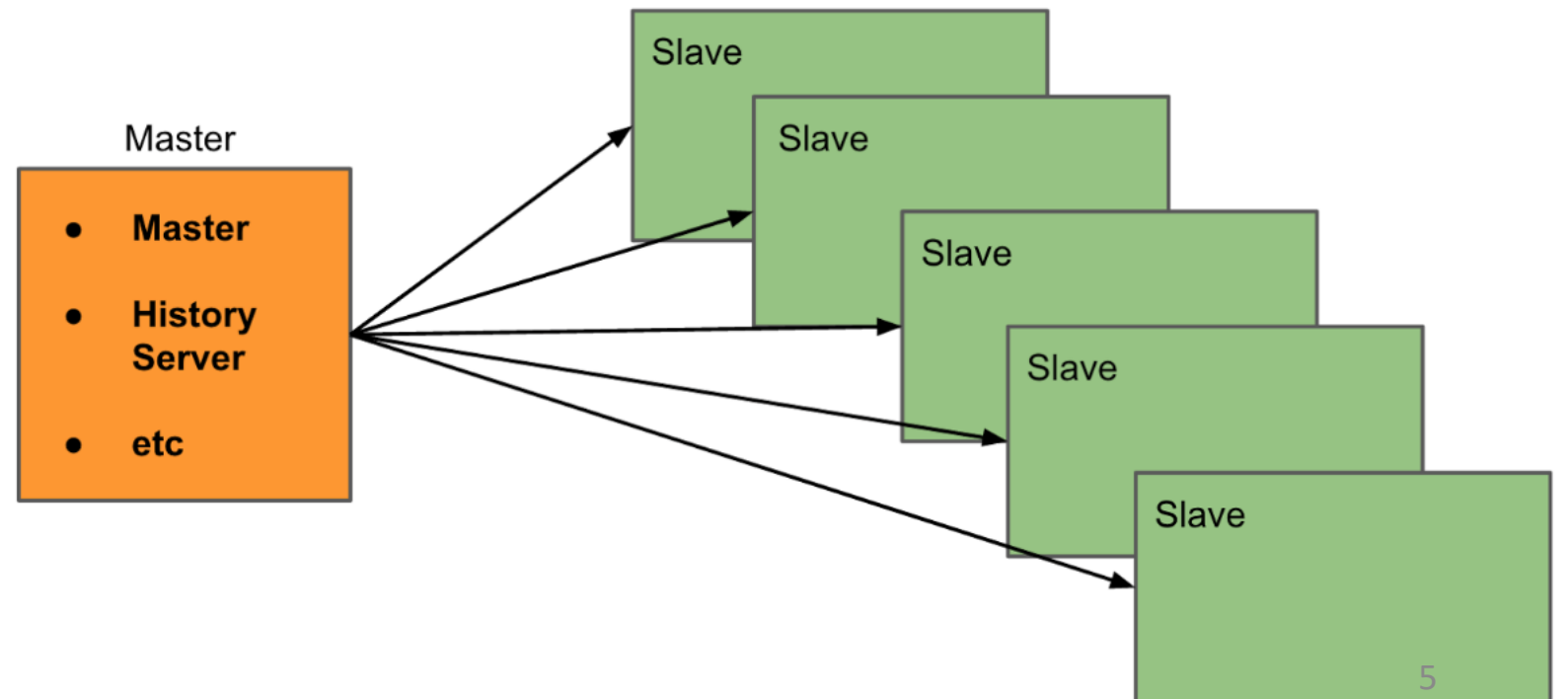
Resilient Distributed Datasets

- Collection of objects (records) that act as one unit
- Stored in main memory or disk
- Parallel operations built on top of them
- Have fault tolerance without replication (lineage)

RDD: Concept



- RDD is read-only
- Distributed either in main memory or disk (automatically decided)



RDD vs. Traditional Shared Memory

Aspect	RDDs	Distr. Shared Mem.
Reads	Coarse- or fine-grained	Fine-grained
Writes	Coarse-grained	Fine-grained
Consistency	Trivial (immutable)	Up to app / runtime
Fault recovery	Fine-grained and low-overhead using lineage	Requires checkpoints and program rollback
Straggler mitigation	Possible using backup tasks	Difficult
Work placement	Automatic based on data locality	Up to app (runtimes aim for transparency)
Behavior if not enough RAM	Similar to existing data flow systems	Poor performance (swapping?)

Why do we need RDD in Spark?

- The key motivations behind the concept of RDD are-
 - Iterative algorithms.
 - Interactive data mining tools
 - **DSM** (Distributed Shared Memory) is a very general abstraction, but this generality makes it harder to implement in an efficient and fault tolerant manner on commodity clusters. Here the need of RDD comes into the picture.
 - In distributed computing system data is stored in intermediate stable distributed store such as [HDFS](#) or Amazon S3. This makes the computation of job slower since it involves many IO operations, replications, and serializations in the process.

Why do we need RDD in Spark?(1)

- The main challenge in designing RDD is defining a program interface that provides fault tolerance efficiently.
- To achieve fault tolerance efficiently, RDDs provide a restricted form of shared memory, based on **coarse-grained transformation** rather than **fine-grained** updates to shared state.
- Spark exposes RDD through language integrated API.
- In integrated API each data set is represented as an object and transformation is involved using the method of these objects.

Features of Spark RDD

- In-memory Computation
- Lazy Evaluations
- Fault Tolerance
- Immutability
- Partitioning
- Persistence
- Location-Stickiness

In-memory Computation

- Spark RDDs have a provision of in-memory computation. It stores intermediate results in distributed memory(RAM) instead of stable storage(disk).

Lazy Evaluations

All transformations in Apache Spark are lazy, in that they do not compute their results right away. Instead, they just remember the transformations applied to some base data set.

Fault Tolerance

- Spark RDDs are fault tolerant as they track data lineage information to rebuild lost data automatically on failure. They rebuild lost data on failure using lineage, each RDD remembers how it was created from other datasets (by transformations like a map, join or groupBy) to recreate itself.

Immutability

- Data is safe to share across processes. It can also be created or retrieved anytime which makes caching, sharing & replication easy. Thus, it is a way to reach consistency in computations.

Partitioning

- Partitioning is the fundamental unit of parallelism in Spark RDD. Each partition is one logical division of data which is mutable. One can create a partition through some transformations on existing partitions.

Persistence

- Users can state which RDDs they will reuse and choose a storage strategy for them (e.g., in-memory storage or on Disk).

Coarse-grained Operations

- It applies to all elements in datasets through maps or filter or group by operation.

Location-Stickiness

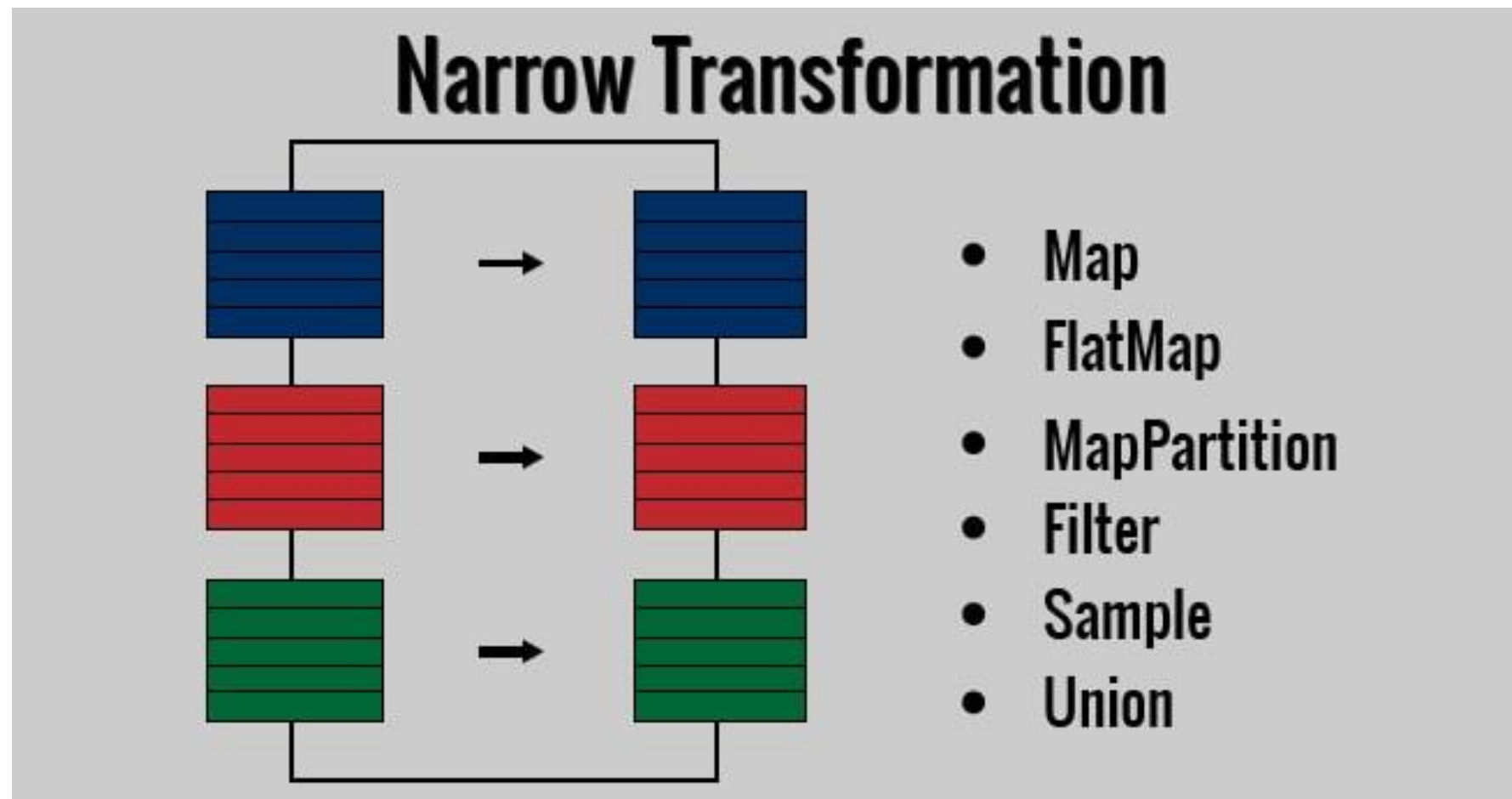
- RDDs are capable of defining placement preference to compute partitions. Placement preference refers to information about the location of RDD. The DAGScheduler places the partitions in such a way that task is close to data as much as possible. Thus, speed up computation

Spark RDD Operations

- Transformation
 - Spark **RDD Transformations** are *functions* that take an RDD as the input and produce one or many RDDs as the output.
 - They do not change the input RDD (since RDDs are immutable and hence one cannot change it), but always produce one or more new RDDs by applying the computations they represent e.g. Map(), filter(), reduceByKey() etc.
 - **Narrow** Transformations
 - **Wide** Transformations
- Actions

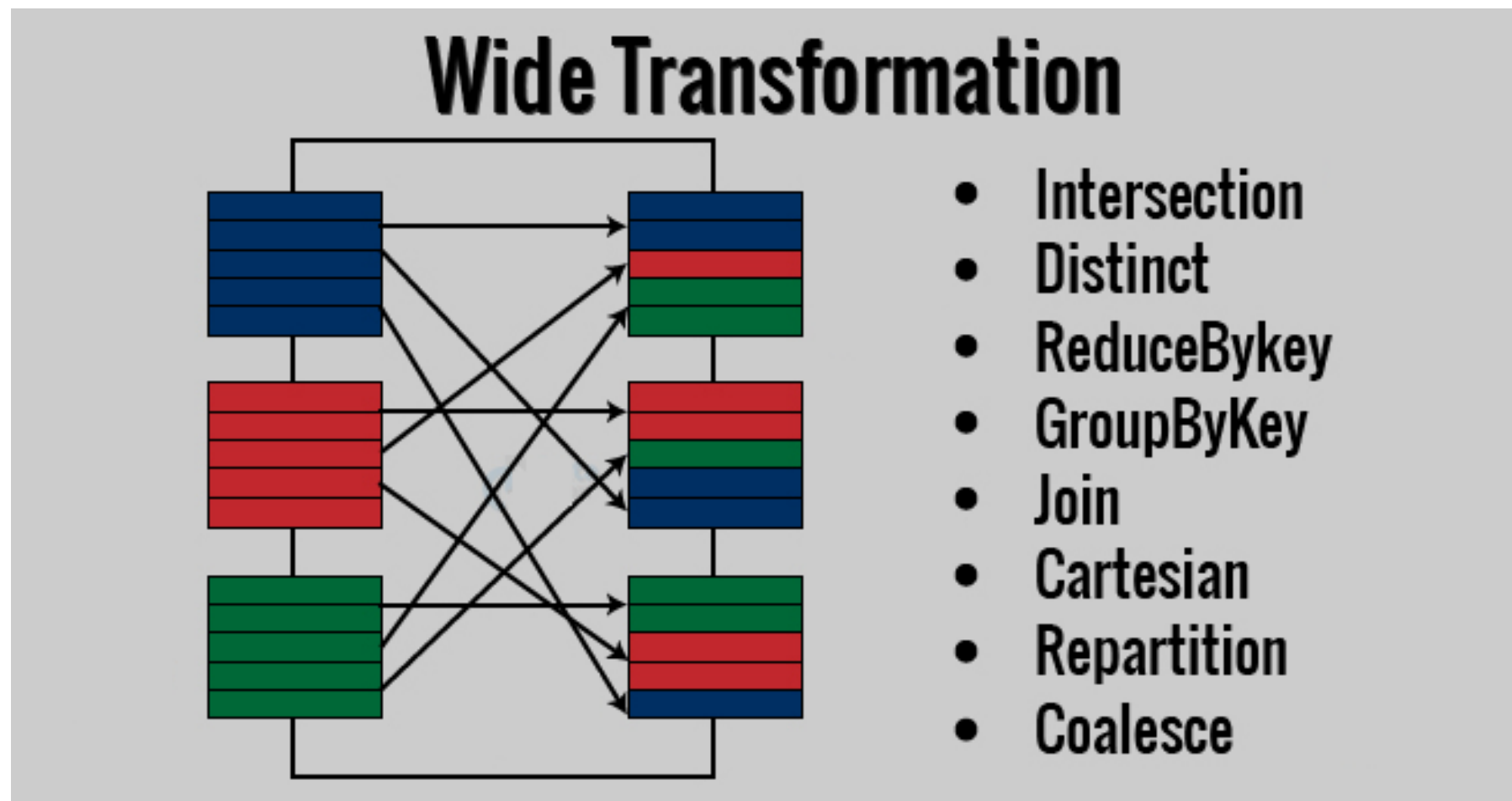
Narrow Transformations

- It is the result of map, filter and such that the data is from a single partition only, i.e. it is self-sufficient. An output RDD has partitions with records that originate from a single partition in the parent RDD. Only a limited subset of partitions used to calculate the result.



Wide Transformations

- It is the result of `groupByKey()` and `reduceByKey()` like functions. The data required to compute the records in a single partition may live in many partitions of the parent RDD. Wide transformations are also known as *shuffle transformations* because they may or may not depend on a shuffle.



Actions

- An **Action** in Spark returns final result of RDD computations.
- It triggers execution using lineage graph to load the data into original RDD, carry out all intermediate transformations and return final results to Driver program or write it out to file system.
- Lineage graph is dependency graph of all parallel RDDs of RDD.
- **Actions** are RDD operations that produce non-RDD values. They materialize a value in a Spark program.
- An Action is one of the ways to send result from executors to the driver. **First(), take(), reduce(), collect(), the count()** is some of the Actions in spark.